The eight fallacies of distributed computing

1. The network is reliable.
    - There will always be packet drops, connection interruptions, and possible data corruption.
2. Latency is zero.
    - Latency is not zero. While internet speed have vastly improved, there will always be delays in sending/receiving messages and this cannot be predicted.
3. Bandwidth is infinite.
    - A design paradox → to reduce latency, we transfer more data to reduce network communication between two PCs. E.g. one-time, big-time send of all the necessary data for processing. But bandwidth is finite! Transferring more data, consumes more bandwidth.
    - Must address and find the balance between #2 and #3 fallacies.
4. The network is secure;
    - Ensure your data is encrypted. Any user could possibly hijack your system. E.g. SQL injection.



5. Topology doesn't change.
    - What if your host/domain changes? Moving from a new server location. If IPs are hardcoded, then all clients must have updated IP directories.

- What if one of the services fail? How do we route incoming connections? E.g. MMO -> Login system working, but item store services are not → should still be able to play the game, minus access to the online shop.

6. There is one administrator;
   - There can be many teams/developers for each service.
7. Transport cost is zero.
   - Related to #2. There will always be a cost in transporting data. E.g. Read/write operations in server disks, usage of virtual machines, GPU rentals, etc. Goal is to maximize usage, minimize cost.
8. The network is homogeneous.
   - The network is heterogenous → Many services have different software/hardware configurations. One must standardize the messages being passed across network → XML, JSON, protocol buffers.

EXAMPLES OF HOW TO ADDRESS SOME OF THE FALLACIES

Addressing #1: Promoting network reliability
Context: Without considering timeout/error handling, a client requesting a service from a server can be assumed to wait forever.

Refer to GDPARCM Sockets Demo. This example doesn't support any timeout mechanism and assumes an indefinite connection between server-client.

In gRPC, any RPC call has a default timeout. The call returns a failure if no response from server was received. The default timeout, per client request is 1 second. To modify/extend the timeout, call **set_deadline()** before making any RPC call.

```
context.set_deadline(std::chrono::system_clock::now() + std::chrono::seconds(5));
result = stub_->CreateFolder(&context, request);
```

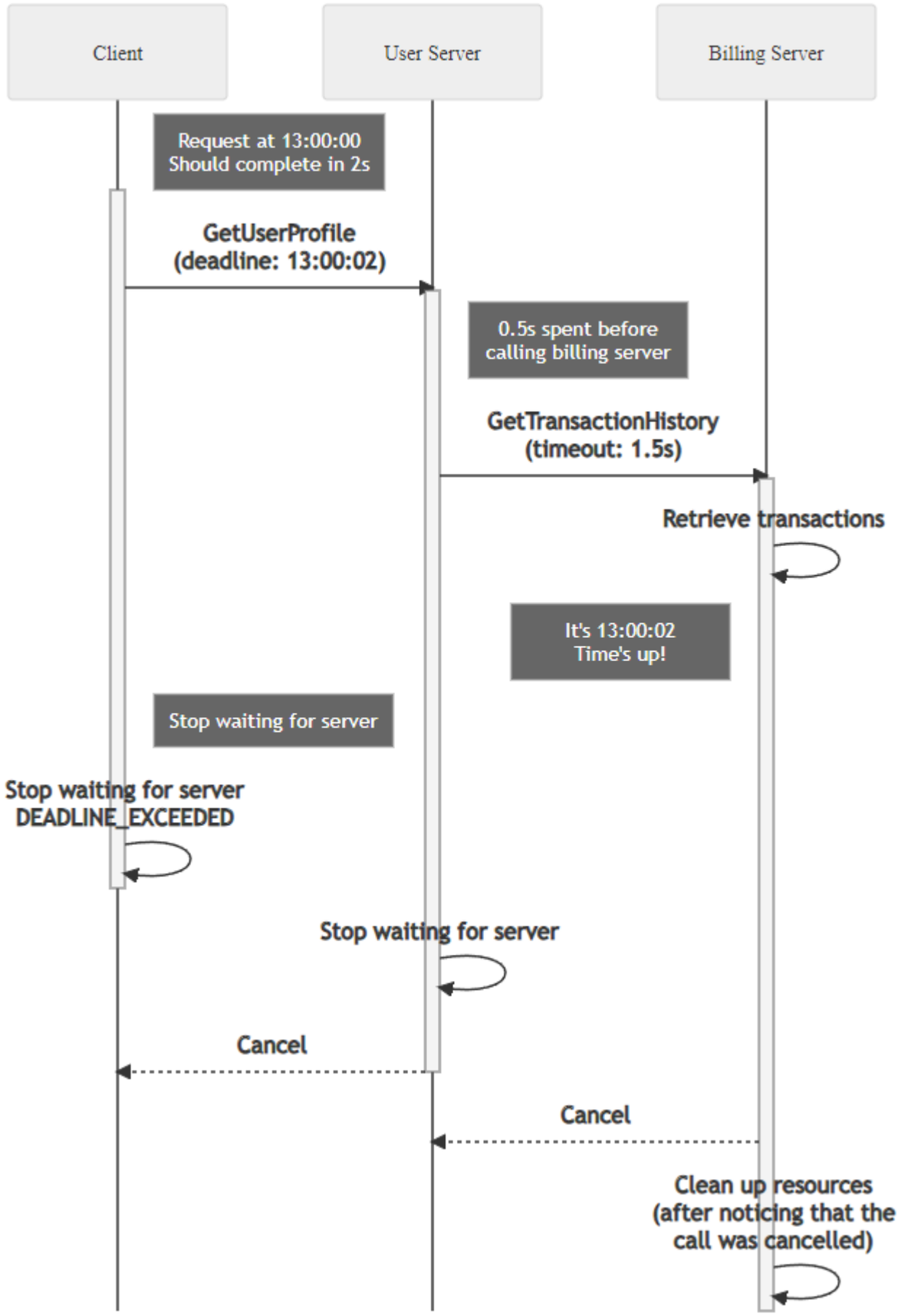
```
10 std::string GreeterClient::SayHello(const std::string& user)
11 {
12     // Follows the same pattern as SayHello.
13     HelloRequest request;
14     request.set_name(user);
15     HelloReply reply;
16     grpc::ClientContext context;
17
18     std::chrono::time_point deadline = std::chrono::system_clock::now() +
19         std::chrono::milliseconds(2000);
20     context.set_deadline(deadline);
21
22     // Here we can use the stub's newly available method we just added.
23     grpc::Status status = stub_->SayHello(&context, request, &reply);
24     if (status.ok()) {
25         return reply.message();
26     }
27     else {
28         std::cout << status.error_code() << ": " << status.error_message()
29             << std::endl;
30         return "RPC failed";
31     }
32 }
```

Documentation: https://grpc.io/blog/deadlines/

- Servers could also set deadlines.
- A server might receive requests from a client with an unrealistically short deadline → <1 second deadline, but requires a 100MB file. To address this, the server application has the last say whether or not to proceed with the request.
  - A gRPC server automatically cancels a call once a deadline set by the client has passed (CANCELLED status).
  - The server application is responsible for stopping any activity it has spawned to service the client request → periodically check if the request initiated has passed the deadline (CANCELLED status).

Deadline Propagation
Your server might need to call another server to produce a response. In these cases where your server also acts as a client you would want to honor the deadline set by the original client. Automatically propagating the deadline from an incoming request to an outgoing one is supported by some gRPC implementations. In some languages this behavior needs to be explicitly enabled (e.g. C++) and in others it is enabled by default (e.g. Java and Go). Using this capability lets you avoid the error-prone approach of manually including the deadline for each outgoing RPC.

Since a deadline is set point in time, propagating it as-is to a server can be problematic as the clocks on the two servers might not be synchronized. To address this gRPC converts the deadline to a timeout from which the already elapsed time is already deducted. This shields your system from any clock skew issues.



Retry Policy
A common solution is to implement a **retry policy →,** periodically request for service to the server, then provide a graceful failure handling strategy if numerous retries are made.

```
---------------------------
Create folder with retry

#include <iostream>
#include <grpcpp/grpcpp.h>
#include "remote_operations.grpc.pb.h"

class RemoteOperationsClient {
public:
    RemoteOperationsClient(std::shared_ptr<grpc::Channel> channel)
        : stub_(RemoteOperations::NewStub(channel)) {}

    Result CreateFolderWithRetry(const std::string& folder_name, int max_retries = 3) {
        FolderRequest request;
        request.set_folder_name(folder_name);

        grpc::ClientContext context;
        Result result;

        for (int attempt = 1; attempt <= max_retries; ++attempt) {
            // Attempt to create a folder
            result = stub_->CreateFolder(&context, request);

            if (result.success()) {
                // Folder creation succeeded
                std::cout << "Folder creation succeeded." << std::endl;
                break;
            } else {
                // Folder creation failed, retry after a delay
                std::cerr << "RPC failed (attempt " << attempt << "/" << max_retries << "): " << result.message() << std::endl;
                std::this_thread::sleep_for(std::chrono::seconds(1));  // Add a delay before retrying
            }
        }

        return result;
    }

private:
    std::unique_ptr<RemoteOperations::Stub> stub_;
};

int main() {
    RemoteOperationsClient client(grpc::CreateChannel("localhost:50051", grpc::InsecureChannelCredentials()));

    // Example: Create a folder with retries
    Result folder_creation_result = client.CreateFolderWithRetry("new_folder");

    if (folder_creation_result.success()) {
        std::cout << "Folder creation succeeded." << std::endl;
    } else {
        std::cerr << "Folder creation failed: " << folder_creation_result.message() << std::endl;
    }

    return 0;
}
```

Addressing #3: Assuming bandwidth is finite, make message buffers and service requests as small as possible.

An example of a violation of #3 – a mini-Spotify web player. Use case: Get a list of user songs and their MP4 files all at once.

```
//Example: violation of #3  ("Bandwidth Is Infinite")

syntax = "proto3";

message Song {
  string title = 1;
  string artist = 2;
  bytes audio_data = 3; // Large binary audio data
}

//instance of a Song
Song song {
  title: "Song Title"
  artist: "Song Artist"
  audio_data: <large_binary_audio_data>
}
```

- The Song message includes an **audio_data** field of type bytes.
- This field represents potentially large binary audio data associated with each song.

- The **<large_binary_audio_data>** represents a substantial amount of binary audio information.

- If this field is transmitted frequently between the client and server without proper optimization, it could lead to increased bandwidth usage and potential performance issues, violating the fallacy that "Bandwidth Is Infinite."

Possible solutions:

1. **Streaming:** Stream the audio data separately from other fields if it's not always needed.
2. **Compression:** Compressing the audio data before transmission to reduce its size.
3. **Caching:** Using caching mechanisms to avoid unnecessary repeated transfers of the same audio data.
4. **External Storage:** Storing large binary data externally (e.g., in a cloud storage service) and transmitting only references or URLs in the protobuf message.

Sample solution using gRPC bidirectional streaming approach:

```
//Addressing #3
syntax = "proto3";

message Song {
  string title = 1;
  string artist = 2;
  string audio_reference = 3;  // Reference or URL to the audio file
}

service MusicService {
  rpc StreamSongs(stream SongRequest) returns (stream Song);
}

message SongRequest {
  repeated string song_ids = 1;
}
```

Server implementation:

```
using grpc::Server;
using grpc::ServerBuilder;
using grpc::ServerContext;
using grpc::ServerWriter;
using grpc::ServerReader;
using grpc::Status;

class MusicServiceImpl final : public MusicService::Service {
public:
    Status StreamSongs(ServerContext* context, ServerReader<SongRequest>* reader, ServerWriter<Song>* writer) override {
        SongRequest song_request;
        while (reader->Read(&song_request)) {
            for (const std::string& song_id : song_request.song_ids()) {
                Song song;
                if (GetSongById(song_id, &song)) {
                    writer->Write(song);
                }
            }
        }
        return Status::OK;
    }

private:
    bool GetSongById(const std::string& song_id, Song* song) {
        // Fetch song details and set the audio_reference field
        // In a real-world scenario, you might retrieve this information from a database or external storage.
        song->set_title("Song Title");
        song->set_artist("Song Artist");
        song->set_audio_reference("https://example.com/audio/song123.mp3");
        return true;
    }
};

void RunServer() {
    std::string server_address("localhost:50051");
    MusicServiceImpl service;

    ServerBuilder builder;
    builder.AddListeningPort(server_address, grpc::InsecureServerCredentials());
    builder.RegisterService(&service);

    std::unique_ptr<Server> server(builder.BuildAndStart());
    std::cout << "Server listening on " << server_address << std::endl;
    server->Wait();
}

int main() {
    RunServer();
    return 0;
}
```

Client implementation:

```cpp
//Client code
#include <iostream>
#include <grpcpp/grpcpp.h>
#include "music_service.grpc.pb.h"

using grpc::Channel;
using grpc::ClientContext;
using grpc::ClientReader;
using grpc::Status;

class MusicServiceClient {
public:
    MusicServiceClient(std::shared_ptr<Channel> channel)
        : stub_(MusicService::NewStub(channel)) {}

    void StreamSongs(const std::vector<std::string>& song_ids) {
        SongRequest song_request;
        for (const std::string& song_id : song_ids) {
            song_request.add_song_ids(song_id);
        }

        ClientContext context;
        std::unique_ptr<ClientReader<Song>> reader(stub_->StreamSongs(&context, song_request));

        Song song;
        while (reader->Read(&song)) {
            // Process each received song
            std::cout << "Received song - Title: " << song.title() << ", Artist: " << song.artist() << ", Audio Reference: " << song.
            audio_reference() << std::endl;
        }

        Status status = reader->Finish();
        if (!status.ok()) {
            std::cerr << "Error streaming songs: " << status.error_message() << std::endl;
        }
    }

private:
    std::unique_ptr<MusicService::Stub> stub_;
};

int main() {
    MusicServiceClient client(grpc::CreateChannel("localhost:50051", grpc::InsecureChannelCredentials()));

    // Example: Stream songs by providing a list of song IDs
    std::vector<std::string> song_ids = {"song123", "song456", "song789", ..., "song9999"};
    client.StreamSongs(song_ids);

    return 0;
}
```

Explanation:

- The Song message now includes an **audio_reference** field, which holds a reference or URL to the audio file.
- The **MusicService** service introduces the **StreamSongs** bidirectional streaming RPC, allowing the client to request and receive songs in chunks.
- The server's **StreamSongs** implementation reads the **SongRequest** from the client, fetches song details (including audio references), and streams the songs back to the client.
- The client's **StreamSongs** method sends a list of song IDs to the server and receives songs in chunks, processing each song as it arrives.

This approach allows the client and server to efficiently exchange references or URLs to audio files, reducing the impact on bandwidth and improving the efficiency of data transfer.

RECOVERY

- Attempts to addresses #1 and #3. Constant occurrences of #1 and #3 will cause numerous connection faults which could lead to cascading failures → a client requesting for service #1, but service #1 needs service #2, #2 needs #3,…. #3 needs #N. If as early as service #1, the request isn't successful, failures will cascade across the network.
- Context: If **K** clients requests for service #1, but service #1 hasn't successfully processed **K/50** (busy/unavailable), then it is not wise to repeatedly ping service #1 → K clients must reroute/request service on other available servers.

Additional background:

- Consider a P4 scenario.
    - Your AI web viewer hung up and needed to restart, but your server keeps sending back images to the AI web viewer → the web viewer gets bogged down with several requests that cannot be processed.
    - Your 3D model viewer keeps requesting for "scene A" that isn't available yet → processing "scene A" gets further delayed because numerous duplicate requests get sent.

The circuit breaker design pattern

- Inspired by an electrical circuit breaker. A circuit breaker design pattern prevents cascading failures, and promotes rerouting of requests.
- A circuit breaker instance monitors the status of the remote service. When a certain threshold of failures is recorded in the circuit breaker, it "trips/opens," preventing any further requests from being sent to that service.

Circuit breaker class template:

```cpp
class CircuitBreaker {
public:
    CircuitBreaker(int failure_threshold, std::chrono::seconds recovery_time)
        : failure_threshold_(failure_threshold), recovery_time_(recovery_time) {}

    bool IsClosed() const {
        return state_ == State::CLOSED;
    }

    void RecordFailure() {
        consecutive_failures_++;
        if (consecutive_failures_ >= failure_threshold_) {
            Open();
        }
    }

    void RecordSuccess() {
        consecutive_failures_ = 0;
        if (state_ == State::OPEN) {
            Close();
        }
    }

private:
    enum class State {
        CLOSED,
        OPEN,
    };

    void Open() {
        state_ = State::OPEN;
        open_time_ = std::chrono::system_clock::now();
    }

    void Close() {
        state_ = State::CLOSED;
    }

    int failure_threshold_;
    std::chrono::seconds recovery_time_;
    int consecutive_failures_ = 0;
    State state_ = State::CLOSED;
    std::chrono::system_clock::time_point open_time_;
};
```

Sample usage. Server using circuit breaker instance:

```cpp
class RemoteOperationsServiceImpl final : public RemoteOperations::Service {
public:
    RemoteOperationsServiceImpl(CircuitBreaker& circuit_breaker)
        : circuit_breaker_(circuit_breaker) {}

    grpc::Status CreateFolder(grpc::ServerContext* context, const FolderRequest* request, Result* response) override {
        if (circuit_breaker_.IsClosed()) {
            // Simulate a failure for demonstration purposes
            circuit_breaker_.RecordFailure();
            return grpc::Status(grpc::StatusCode::UNAVAILABLE, "Service unavailable");
        } else {
            // Simulate a successful operation
            circuit_breaker_.RecordSuccess();
            response->set_success(true);
            response->set_message("Folder created successfully");
            return grpc::Status::OK;
        }
    }

private:
    CircuitBreaker& circuit_breaker_;
};

void RunServer() {
    std::string server_address("0.0.0.0:50051");
    CircuitBreaker circuit_breaker(3, std::chrono::seconds(10)); // Open circuit after 3 consecutive failures, recover after 10 seconds
    RemoteOperationsServiceImpl service(circuit_breaker);

    grpc::EnableDefaultHealthCheckService(true);
    grpc::reflection::InitProtoReflectionServerBuilderPlugin();
    grpc::ServerBuilder builder;
    builder.AddListeningPort(server_address, grpc::InsecureServerCredentials());
    builder.RegisterService(&service);
    builder.RegisterService(grpc::reflection::CreateProtoReflectionService());
    std::unique_ptr<grpc::Server> server(builder.BuildAndStart());
    std::cout << "Server listening on " << server_address << std::endl;
    server->Wait();
}

int main() {
    RunServer();
    return 0;
}
```

- The **CircuitBreaker** class tracks failures and decides whether the circuit should be open or closed.
- The **RemoteOperationsServiceImpl** class simulates a gRPC service that may fail under certain conditions.
- When the circuit breaker is open, the service returns an **UNAVAILABLE** status, simulating a failure.
- The circuit breaker records failures and successes based on the service's responses.

More sophisticated and dedicated libraries are available: Netflix Hystrix, Resilience4j, Sentinel.

C++ libraries

- Cpp-httplib: https://github.com/yhirose/cpp-httplib
- Envoy: https://www.envoyproxy.io/

Advantages of circuit breaker

- Efficient way to halt future client requests that are guaranteed to fail if the service is unavailable.
- Opportunity to reroute requests → circuit breaker #1 trips → reroute K clients to another service (with an associated circuit breaker #2).
- Allows unavailable services to focus their computing resources for restarting. E.g. rebooting.

Disadvantages of circuit breaker

- Complexity → N services = N circuit breakers?
- Configuration-dependent and educated guessing of thresholds → timeouts, recovery times, number of failures require careful tuning to maintain the quality of the underlying services.
    - Overly aggressive thresholds lead to false positives, causing the system to unnecessarily go into a degraded state.
- Overhead → a circuit breaker is typically a running thread of its own.